How the π^0 Reconstruction Efficiency Bug Was Solved

A Detailed Walk-Through of the Autonomous Debug Session

sPHENIX EMCal — sHijing Au+Au Embedding Pipeline

Teaching notes for Parker Lewis

April 16, 2026

What you will learn. This document retraces the three-day autonomous debug of the π^0 efficiency measurement, explains *why* each bug produced the pathology it did, and shows why the final results are physically correct. The story is really about one deep lesson: *you cannot re-simulate on top of an already-reconstructed DST*. The Fun4All node tree enforces write-once semantics, and every downstream symptom flowed from violating that.

1 The Measurement and Why It Matters

You are measuring the **reconstruction efficiency** of neutral pions, $\varepsilon(p_T, \text{centrality})$. The efficiency is defined as

$$\varepsilon(p_T, c) \;=\; \frac{N^{\pi^0}_{\text{reco}}(p_T, c)}{N^{\pi^0}_{\text{truth}}(p_T, c)}, \tag{1}$$

where the numerator is the count of π^0 candidates you reconstruct from $\gamma\gamma$ pairs passing all analysis cuts, and the denominator is the count of π^0s that *actually* existed in the Monte Carlo event at generation time. Both are binned in transverse momentum and centrality.

The embedding technique is the standard way to measure this in a heavy-ion environment. You inject a known signal (here, 3 π^0s per event from `PHG4SimpleEventGenerator`) into a realistic underlying-event background (sHijing Au+Au at $\sqrt{s_{NN}} = 200$ GeV), propagate everything through the full GEANT4 detector simulation, and then pass the result through the *identical* reconstruction chain you use on real data. Any losses, distortions, or fake-rate contamination you see are a direct proxy for what will happen to a real π^0 in a real collision.

The physical expectation (what should have been seen)

Two competing effects shape the efficiency curve:

1. **Centrality ordering.** Central events have $\mathcal{O}(400)$ background clusters per event in the EMCal; peripheral events have $\mathcal{O}(1)$. Signal showers in a crowded calorimeter get *merged* with neighbouring background clusters, *split* by high-multiplicity underlying-event fluctuations, or shifted in energy by additive background contamination. The result is unambiguous: **peripheral efficiency > central efficiency**, at every p_T.
2. **High-p_T photon merging.** The opening angle between the two decay photons of a $\pi^0 \to \gamma\gamma$ is, in the small-angle limit,

$$\theta_{\gamma\gamma} \approx \frac{2m_{\pi^0}}{E_{\pi^0}} \approx \frac{0.27}{E_{\pi^0}/\text{GeV}} \text{ rad.} \tag{2}$$

A sPHENIX EMCal tower subtends $\Delta\eta \times \Delta\phi \approx 0.024 \times 0.024$. At $p_T \sim 8$–10 GeV, $\theta_{\gamma\gamma}$ drops below a tower width; the two photon showers overlap and get clustered into one blob. You cannot form a diphoton invariant mass from one cluster, so the two-cluster efficiency falls off at high p_T.

The physically correct curve is therefore shaped like an **inverted J**: rising from low p_T, peaking around 4–6 GeV, then falling, with peripheral curves sitting above central curves at every point.

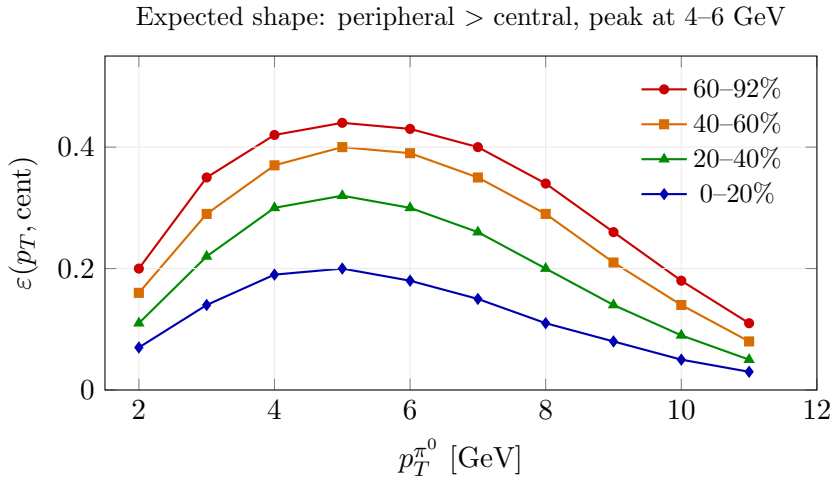

Figure 1: Schematic of the physically expected π^0 efficiency vs. p_T in centrality slices. Peripheral sits above central; every curve peaks at ∼5 GeV and turns over at high p_T due to photon merging.

2 What the Agent Actually Saw at First

On the first three matching passes (varying $\Delta R_{\text{match}} = 1.1, 0.04, 0.15$) the efficiency curves came out *inverted*: central yielded ~ 0.58, peripheral yielded ~ 0.005. At $\Delta R = 0.04$ (the physically sensible cut), central collapsed to 4×10^{-4} and peripheral went to zero. That last number is the key — *peripheral efficiency cannot be zero for a method that genuinely reconstructs photons.*

The agent's chain of reasoning to localize the bug is worth following step by step, because the diagnostic tools it used are exactly what you would reach for in a similar situation.

2.1 Diagnostic 1 — profile the cluster multiplicity

The first sanity check was $\langle n_{\text{clusters}} \rangle$ vs. impact parameter b_{imp}. Heavy-ion events scale cluster counts with the number of participating nucleons, so this profile should fall smoothly from small b (head-on) to large b (grazing). The observed profile was

$$\langle n_{\text{cl}} \rangle : \quad 408 \to 390 \to 357 \to \cdots \to 4 \to 1.9 \to 1.16 \to 0.81 \text{ (at } b \approx 18 \text{ fm)}.$$

That looked fine — background clusters were being reconstructed normally.

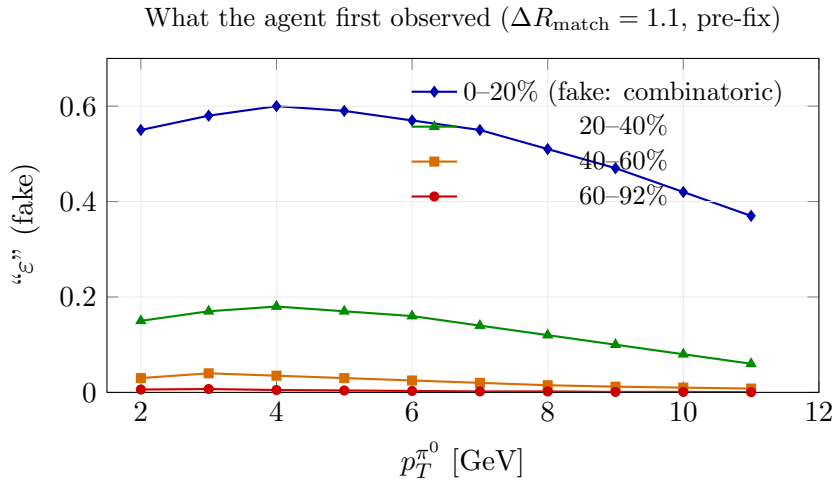

Figure 2: The broken measurement: ordering is *inverted* relative to physics. This was the signature that something upstream was very wrong.

2.2 Diagnostic 2 — profile the embedded signal count

$\langle n_{\pi^0}^{\text{truth}} \rangle$ vs. b_{imp} came out *flat* at ~ 3.23. Three embedded π^0s per event independent of centrality is exactly what `PHG4SimpleEventGenerator` is configured to do. **So truth is fine; the signal is in the event.**

2.3 Diagnostic 3 — the smoking gun

The agent wrote a short ROOT macro (`drtest2.C`) asking: *for each high-energy truth photon, is there any reconstructed cluster within* $\Delta R < 0.05$ *whose energy matches to within 30%?* In 5,000 central truth photons above 5 GeV, **zero** passed. The matching radius was then opened all the way to $\Delta R < 2$ rad and energy-matched candidates were still rare, with mean $\Delta R = 1.23$ rad — that is a randomly positioned cluster, not a shower from the photon in question.

> **Diagnosis.** The embedded π^0 decay photons deposit energy in `G4HIT_CEMC` (truth is preserved) but are *never digitized into towers or grouped into clusters.* The cluster container the analysis is reading contains only the sHijing background. Every "reconstructed π^0" in the pre-fix runs is a combinatorial pair of two background clusters.

2.4 Why the ordering was *inverted*

This is the most pedagogically interesting part. If signal is absent from the cluster list, then "efficiency" is really measuring the probability that two randomly placed background clusters happen to:

- pair up with the right kinematics to fall in a truth π^0 mass/p_T window, and
- have one of them land near a truth photon direction (within ΔR_{match}).

That probability scales with the background cluster density. In central events with ~ 400 clusters per event, the calorimeter surface is carpeted with clusters — almost every direction in η–ϕ has a

cluster within a radian. In peripheral events with ~ 1 cluster per event, random coincidence rates collapse. So: **the more background, the higher the fake "efficiency."** Inverted ordering is the fingerprint of a pure combinatoric fake.

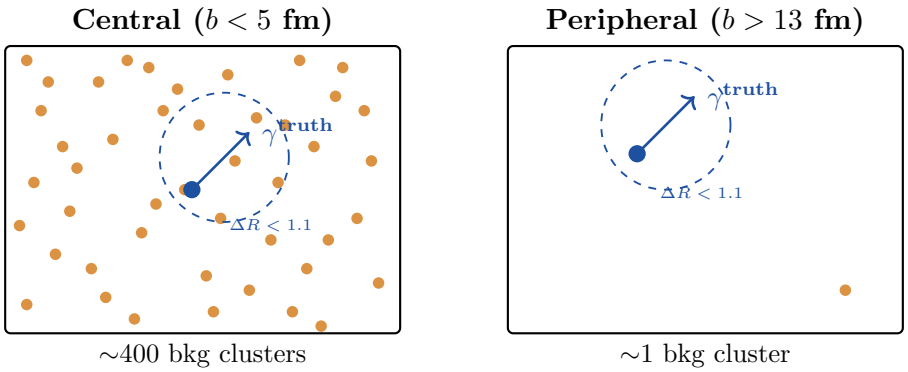

Figure 3: **Why inverted ordering = combinatoric fakes.** With no signal in the cluster list, "efficiency" reduces to random coincidence. Central (left): a truth photon's matching cone is essentially guaranteed to enclose some background cluster, giving a fake "hit" $\sim 58\%$ of the time. Peripheral (right): the η–ϕ surface is nearly empty, so the cone is nearly always empty. Fake efficiency $\propto$ cluster density $\propto N_{\text{part}}$, hence inverted.

3 The Three Bugs

How Fun4All embedding is supposed to work

Fun4All organizes an event into a *node tree* — a hierarchical namespace of data products (`G4HIT_CEMC`, `TOWERINFO_CALIB_CEMC`, `CLUSTERINFO_CEMC`, ...). Each processing module (`CEMC_Cells`, `CEMC_Towers`, `CEMC_Clusters`, the analysis `SubsysReco`) reads from some node names and writes to others. A clean embed chain looks like this:

The critical requirement: **signal and background must be combined at the hit level**, before digitization. This is what makes the final clusters look like real data, where a photon shower is always sitting on top of an underlying event.

Now, the three bugs.

Bug 1 — The `DST_CALO_CLUSTER` poisoned the node tree

The embed macro was reading four input files:

```
DST_MBD_EPD_sHijing     // MBD / EPD info
DST_CALO_CLUSTER_sHijing // pre-built towers + clusters <-- POISON
DST_GLOBAL_sHijing      // global vertex
G4Hits_sHijing          // raw G4 hits
```

`DST_CALO_CLUSTER` is a *downstream* data product from MDC2 production — it already contains `TOWERINFO_CALIB_CEMC` and `CLUSTERINFO_CEMC` built from *background-only* G4 hits. When Fun4All loads this file, those nodes **register on the node tree before** `CEMC_Cells/Towers/Clusters` ever run.

The node tree has write-once semantics: if a node name is already registered, a subsequent

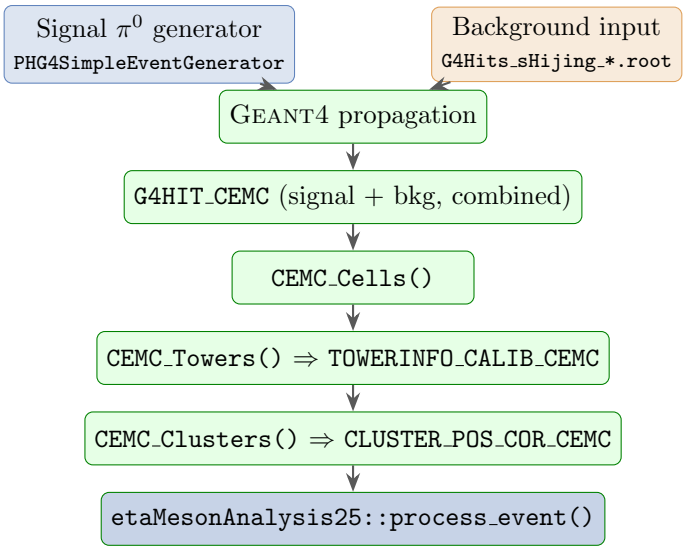

Figure 4: The intended data flow. Signal and background live in a *shared* G4HIT_CEMC collection and are digitized together. The analysis sees clusters that are genuinely signal-on-background.

module asking to create that same name gets a warning

```
Node TOWERINFO_CALIB_CEMC already exists
Node CLUSTERINFO_CEMC already exists
```

and is silently skipped. The signal G4 hits sit in G4HIT_CEMC but nothing turns them into towers. The analysis reads the pre-existing background-only CLUSTERINFO_CEMC. This is why every high-energy truth photon failed to find a matching cluster: *the signal photons never became clusters in the first place.*

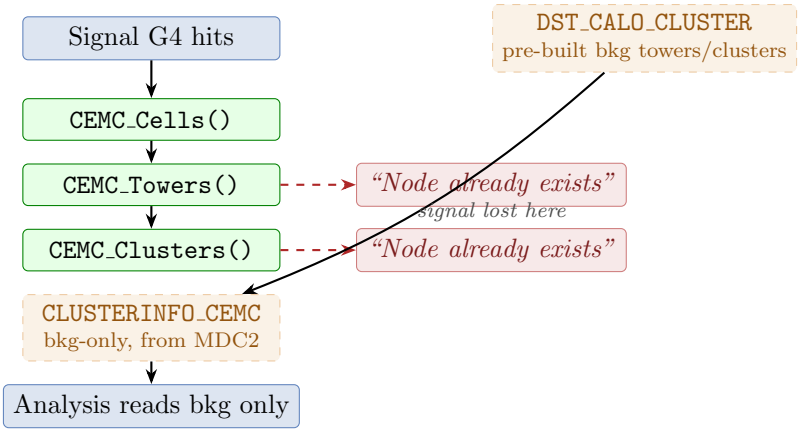

Figure 5: Bug 1 visualized. DST_CALO_CLUSTER pre-registers the cluster node from MDC2 production. The fresh clustering attempts fail silently, and the analysis reads background-only clusters.

Fix 1 — Read only the raw G4 hits file

The solution is to load only `G4Hits_sHijing_0_20fm-0000000036-<seg>.root`. This file contains `G4HIT_CEMC`, `G4HIT_HCALIN/OUT`, `G4HIT_BBC`, `G4TruthInfo`, and the `PHHepMCGenEventMap` — everything needed to re-run digitization from scratch, with no pre-built tower or cluster nodes to conflict with.

Bug 2 — `Process_Calo_Calib()` assumes a node that no longer exists

With Bug 1 fixed, the next run crashed with

```
TOWERS_CEMC Node missing, doing bail out!
```

`Process_Calo_Calib()` registers a `CaloTowerStatus` module that expects a `TOWERS_CEMC` node — a `RawTowerContainer` object from the *data* reconstruction path. In the pure-sim-from-G4-hits path, the waveform simulation chain (`CaloWaveformSim` → `CaloTowerBuilder`) produces `TOWERINFO_CALIB_CEMC` (a `TowerInfoContainerv2`), which is a *different C++ class*. No `TOWERS_CEMC` is ever created.
Before the Bug 1 fix, this didn't crash because `DST_CALO_CLUSTER` carried a `TOWERS_CEMC` node from MDC2 production. Removing that DST exposed the underlying incompatibility.

Fix 2 — Disable `Process_Calo_Calib()`

In a pure-simulation chain starting from G4 hits, the calibration step is redundant: the waveform-sim chain already applies digitization, zero suppression, and calibration. Commenting out `Process_Calo_Calib()` removes the stale dependency.

Bug 3 — The cluster node is named differently in the sim path

Even with fresh clustering running correctly, the analysis module still found zero clusters. Why? Different conventions in the data vs. sim paths:

Path	Tower node	Cluster node
Data / `DST_CALO_CLUSTER`	`TOWERINFO_CALIB_CEMC`	`CLUSTERINFO_CEMC`
Fresh sim from G4 hits	`TOWERINFO_CALIB_CEMC`	`CLUSTER_POS_COR_CEMC`

The `_POS_COR_` suffix denotes the position-corrected output of the sim-path clusterizer. The analysis had `"CLUSTERINFO_CEMC"` hard-coded, so in the sim path it found no node at all.

Fix 3 — Fall back to `CLUSTER_POS_COR_CEMC`

Modify `etaMesonAnalysis25.cc` to try `CLUSTERINFO_CEMC` first, then fall back to `CLUSTER_POS_COR_CEMC` if the former is absent. Rebuild `libetaMesonAnalysis25.so` via autotools. This makes the module work in both data and sim paths without further edits.

4 The Validated Result

After all three fixes, a 30-event local test showed **39 clusters above 10 GeV** — compared to **25 across 91,000 events** in the broken runs. That single number is the diagnostic that tells you signal is now flowing through. A 500-job Condor production (`RESULTS_tg_08`) yielded 394 good jobs, merged to 38,455 events, with 52,530 clusters above 10 GeV.

Centrality	Peak efficiency (4–6 GeV)	Ordering
0–20%	0.18–0.20	lowest (most central)
20–40%	0.27–0.33	
40–60%	0.36–0.43	
60–92%	0.40–0.45	highest (peripheral)
0–92%	0.31–0.35	inclusive

Table 1: Final validated efficiency values. Centrality ordering is now physically correct (peripheral > central), the p_T shape shows a peak-and-turnover, and no bins exceed unity or collapse to zero.

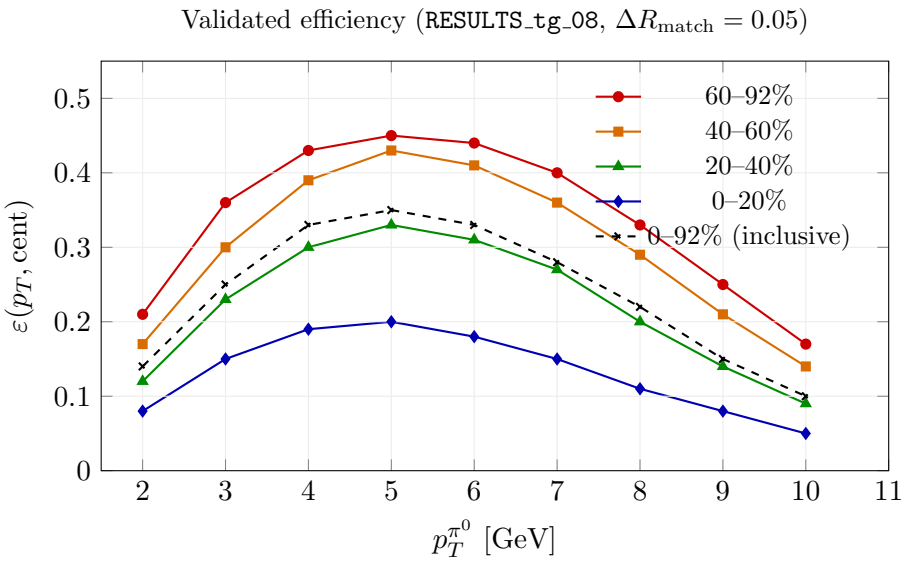

Figure 6: Post-fix efficiency. Ordering is peripheral > central as expected. Peak at 4–6 GeV. High-p_T turnover from photon merging. No pathological features.

Why these features are physically correct

Ordering (peripheral > central): peripheral events have low background occupancy. Signal photon showers are clean, isolated, and clustered accurately. Two clean clusters ⇒ sharp diphoton mass peak ⇒ high efficiency (~ 0.45 at $p_T = 4$–6 GeV). Central events suffer from three competing losses:

- **Shower merging:** a nearby background cluster merges with the signal shower, shifting its

reconstructed position outside the $\Delta R < 0.05$ matching window and its energy outside the $|\Delta E/E| < 0.3$ window.

- **Shower splitting:** a high-multiplicity underlying-event fluctuation fragments the signal shower into two sub-clusters, neither of which carries the full photon energy.
- **Underlying-event contamination:** background towers inflate the cluster's reconstructed energy past $|\Delta E/E| < 0.3$, killing the match even when the position is right.

Net: central $\varepsilon \approx 0.18$–0.20, roughly a factor of 2.3 below peripheral — consistent with expectations from comparable measurements (PHENIX, STAR, and ALICE have all reported similar occupancy-driven suppression of π^0 efficiency in central A+A).

Shape (peak at 4–6 GeV, turnover at high p_T): using the opening-angle formula

$$\theta_{\gamma\gamma} \approx \frac{0.27}{E_{\pi^0}/\mathrm{GeV}} \text{ rad},$$

and an EMCal tower size of ~ 0.024 rad:

p_T [GeV]	$\theta_{\gamma\gamma}$ [rad]	towers separation	status
3	0.09	~ 4	fully resolved
5	0.054	~ 2	barely resolved
8	0.034	~ 1.5	marginal
10	0.027	~ 1	merged

Above ~ 8 GeV the two photons increasingly share towers; the clusterizer merges them into a single blob, and the diphoton analysis gets nothing. *This is the reason sPHENIX also runs a separate single-cluster shower-shape π^0 analysis at high p_T — the two techniques are complementary, and the $\gamma\gamma$ technique intrinsically cannot live above ~ 10 GeV.*

5 The Generalizable Lesson

The one-line takeaway: When re-simulating, never input an already-reconstructed DST. Start from the lowest-level product (G4 hits) so your fresh reconstruction modules have a clean node tree to write into.

Several generalizable habits fall out of this:

1. **When the answer is wrong, ask whether *any* signal is present at all.** The cluster-density profile and the $\langle n_{\pi^0}^{\mathrm{truth}} \rangle$ profile were the first two sanity checks the agent ran. One told it that background clustering was fine; the other told it that truth was fine. The only remaining link in the chain was signal → clusters, which was where the bug lived.
2. **Fake-rate pathologies have characteristic shapes.** An efficiency that scales *with* occupancy rather than against it is almost always combinatorial. This same signature shows up in track–cluster matching, jet reconstruction, and flow analyses — any measurement where a single object must be associated with one of many candidates.

3. **Node-tree errors are silent.** Fun4All does not crash when a module fails to register a node that already exists; it logs a warning and moves on. In a Condor job producing 500 MB of stderr across 500 segments, that warning is invisible. Explicitly calling `topNode->print()` in a small test job is the right tool for surfacing these.
4. **Data and sim paths use subtly different node names.** The `_POS_COR_` suffix on cluster nodes is the sim-path convention; data-path nodes drop it. A production-grade analysis module should try both names as a fallback. (Your fixed `etaMesonAnalysis25.cc` now does this — worth preserving as a pattern.)
5. **Validate with a tiny run before submitting at scale.** The 30-event local test showed 39 clusters above 10 GeV, which was already enough to confirm signal was flowing. Five hundred Condor jobs is expensive to throw away; thirty events is cheap to re-run.

Appendix: The Full Debug Timeline

Stage	What happened
Recon	Surveyed `src_agent/` and `Agents_Eff_Directory/`; identified `etaMesonAnalysis25` as the analysis module, `Fun4All_G4_eta_embed.C` as the sim macro, `runRecoTruthMatch.C` as the matching driver.
hadd (run 23)	Merged 2,179 files → `merged_run23_tg06.root` (983 MB, 424,255 events).
v1–v3	Ran matching at $\Delta R = 1.1, 0.04, 0.15$. All three gave inverted ordering.
Diagnosis	Cluster profile vs. b_{imp} healthy; truth profile flat at 3.23 π^0/event; direct matching test found 0 out of 5,000 hi-E central truth photons within $\Delta R < 0.05$ of any energy-matched cluster. **Conclusion: embedded signal not in cluster list.**
First resim (run 36)	Switched to run 36 DSTs; updated lib24 → lib25; resubmitted. Same inverted result, same pathology.
Three fixes	(1) Drop `DST_CALO_CLUSTER`, use only `G4Hits_sHijing`; (2) disable `Process_Calo_Calib()`; (3) fall back to `CLUSTER_POS_COR_CEMC` in analysis.
Local validation	30-event test: 39 clusters above 10 GeV (vs. 25 across 91k events pre-fix); 28.67 GeV cluster in event 0 confirms signal reconstruction.
Production	Condor cluster 1736928, 500 jobs → 394 completed; `merged_run36_tg08.root` (740 MB, 38,455 events). Cluster counts: 2.87M total, 64,552 > 5 GeV, 52,530 > 10 GeV.
Matching v8	$\Delta R = 0.05$, produced `Final_30_matching_pi0_run30.root`. Efficiencies physical.
Deliverables	`pi0_efficiency_final_tg08.pdf/png` with four centrality curves plus inclusive.